

EDS Lab Assignments

Practical 2

Name: Kesha Devaji Bahekar

PRN: 202501040031

2.1.1 List Operations

Problem Statement:

Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:

1. Add
2. Remove
3. Display
4. Quit

The program should repeatedly prompt the user to enter a choice from the menu.

Depending on the choice selected, the program should perform the following actions:

- **Add:** Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
- **Remove:** Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
- **Display:** Displays the current list of integers. If the list is empty, display "List is empty".
- **Quit:** Exits the program.
- The program should handle invalid menu choices by displaying "Invalid choice". Ensure that the program continues to prompt the user until they choose to quit (option 4).

Code:

```
l1 = []
while True:
    print("1. Add")
    print("2. Remove")
    print("3. Display")
    print("4. Quit")

    n=int(input("Enter choice: "))
    if n==1:
        add = int(input("Integer: "))
        l1.append(add)
```

Output:



2.1.2 Dictionary Operations

Problem Statement:

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Code:

```
# Initial dictionary with 10 predefined records
```

```
student = {  
    1: "Amit",  
    2: "Riya",  
    3: "Kiran",  
    4: "Neha",  
    5: "Arjun",  
    6: "Pooja",  
    7: "Rahul",  
    8: "Sneha",  
    9: "Vikram",  
    10: "Anjali"  
}
```

```
# write your code here..
```

```
print("Original Dictionary:", student)
```

```

new_key = int(input())
new_value = input()
student[new_key] = new_value
print("After Insertion:", student)

```

```

update_key = int(input())
new_val = input()
if update_key in student:
    student[update_key] = new_val
print("After Update:", student)

```

```

del_key = int(input())
if del_key in student:
    del student[del_key]
print("After Deletion:", student)

```

```

print("Traversing Dictionary:")
for key, value in student.items():
    print(f"{key} : {value}")

```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, a file named '2.1.2. Dictionary Operations' is open, containing a Python program that performs insertion, update, deletion, and traversal on a dictionary. The program starts with a predefined dictionary of 10 records. The user is prompted to perform these operations using input. The right panel shows the execution results, including a test case summary indicating that 2 out of 2 shown test cases passed. Below this, the 'Expected output' and 'Actual output' are displayed side-by-side, showing the state of the dictionary after each operation (Insertion, Update, Deletion, and Traversal).

2.1.2. Dictionary Operations

Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.

Operations to be Performed:

1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.

Input and Output Format:

1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.

The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.

Note:

- All operations must be performed using dictionary methods.
- Perform deletion only if possible; leave the dictionary unchanged.
- Refer to the visible test cases for better understanding and strictly match with the input/outputs.

Sample Test Cases

Test Case Summary:

- Average time: 0.034 s
- Maximum time: 0.049 s
- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1

Expected output

```

Original Dictionary: ({1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'})

11
Suresh

After Insertion: ({1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'})

3
Karthik

After update: ({1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'})

After Deletion: ({1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'})

Traversing Dictionary:
1 : Amit
2 : Riya
3 : Karthik
4 : Neha
6 : Pooja
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
11 : Suresh

```

Actual output

```

Original Dictionary: ({1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'})

11
Suresh

After Insertion: ({1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'})

3
Karthik

After update: ({1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'})

After Deletion: ({1: 'Amit', 2: 'Riya', 3: 'Karthik', 4: 'Neha', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali', 11: 'Suresh'})

Traversing Dictionary:
1 : Amit
2 : Riya
3 : Karthik
4 : Neha
6 : Pooja
7 : Rahul
8 : Sneha
9 : Vikram
10 : Anjali
11 : Suresh

```

2.2.1 Linear Search Technique

Problem Statement:

Write a program to check whether the given element is present or not in the array of elements using linear search.

Input format:

- The first line of input contains the array of integers which are separated by space
- The last line of input contains the key element to be searched

Output format:

- If the element is found, print the index.
- If the element is not found, print **Not found**.

Code:

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1  
  
arr = list(map(int,input().split()))  
key = int(input())  
  
result = linear_search(arr, key)  
  
if result != -1:  
    print(result)  
else:  
    print("Not found")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '2.2.1. Linear search Technique' is shown, including the input/output format and a sample test case. The main editor shows the Python code for the linear search algorithm. The right sidebar displays the execution results, indicating that 2 out of 2 shown test cases passed. The test cases show the input array and key, the expected output (index or 'Not found'), and the actual output.

Test Case	Input	Expected Output	Actual Output
Test case 1	1 2 3 4 5 6 3	3	3
Test case 2	1 2 3 4 5 6 7 8 9 10 28	Not found	Not found

2.2.2 Captain of the Team

Problem Statement:

You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.

Input Format:

The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.

Output Format

The output should be the height (in centimeters) of the tallest player.

Code:

```
heights = list(map(int,input().split()))
tallest_player = max(heights)
print(tallest_player)
```

Output:

The screenshot displays the CDETANTRA IDE interface. On the left, the problem statement for '2.2.2 Captain of the Team' is visible, including the input and output formats. The main editor area shows the Python code: `heights = list(map(int,input().split()))`, `tallest_player = max(heights)`, and `print(tallest_player)`. The right sidebar shows the 'Debugger' tab with test case results. The first test case passed, with an expected output of 191 and an actual output of 191. The second test case also passed, with an expected output of 191 and an actual output of 191. The bottom status bar indicates '1 out of 1 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'.